PYTHON PROGRAMMING

CASE STUDY ASSIGNMENT - 2

1. Problem Statement

Warehouse inventory management involves tracking the quantities of different products stored in a warehouse. The opening stock, incoming quantities, and dispatched quantities must be compared with the actual closing stock to identify discrepancies. This project develops a Python program to calculate the expected closing stock for multiple products, compare it with the actual closing stock, identify shortages or surpluses, and calculate replenishment requirements.

2.Objectives

1. To calculate the expected closing stock of multiple products.
2. To compare expected stock with actual closing stock.
3. To identify stock shortages and surpluses.
4. To calculate the replenishment quantity required to reach a target stock level.
5. To maintain accurate inventory records using Python.

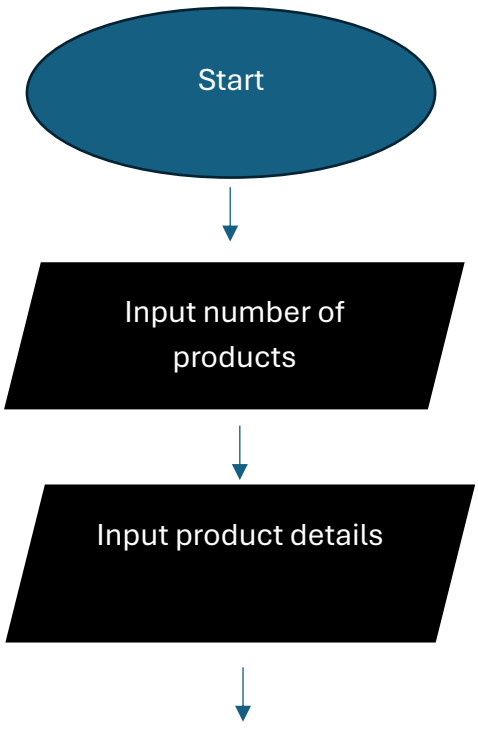
3. Assumptions

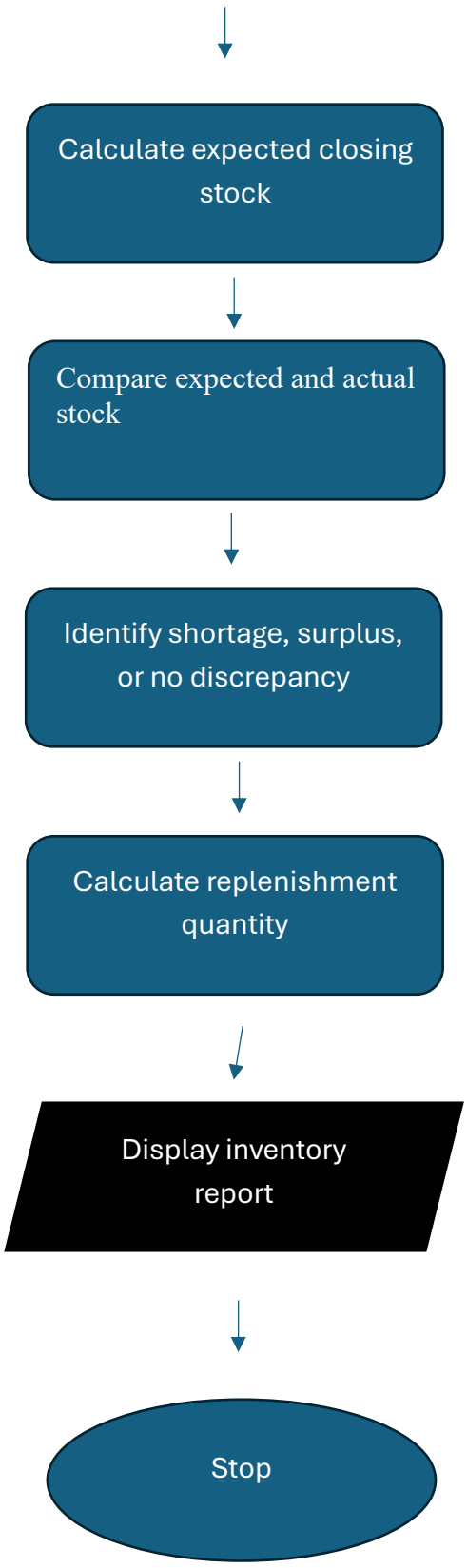
1. Each product has a unique name.
2. Opening stock, incoming quantity, dispatched quantity, and actual closing stock are entered as non-negative whole numbers.
3. Dispatched quantity should not exceed the available stock.
4. Expected closing stock is calculated as opening stock plus incoming quantity minus dispatched quantity.
5. A difference between expected and actual closing stock is treated as a discrepancy.
6. The target stock level is provided by the user to calculate replenishment requirements.

4. Algorithm

1. Start the program.
2. Ask the user to enter the number of products.
3. For each product, input its name, opening stock, incoming quantity, dispatched quantity, actual closing stock, and target stock level.
4. Calculate the expected closing stock using the formula:
5. Expected Closing Stock = Opening Stock + Incoming Quantity − Dispatched Quantity
6. Compare the expected closing stock with the actual closing stock.
7. If both values are equal, display "No Discrepancy".
8. If actual closing stock is less than expected closing stock, calculate and display the shortage.
9. If actual closing stock is greater than expected closing stock, calculate and display the surplus.
10. Calculate the replenishment quantity required to reach the target stock level.
11. Display the inventory report for each product.
12. Stop the program.

5.Flowchart




Calculate expected closing stock
Compare expected and actual stock
Identify shortage, surplus, or no discrepancy
Calculate replenishment quantity
Display inventory report
Stop

6. Pseudocode

```
START
INPUT number_of_products
FOR each product
   INPUT product_name
   INPUT opening_stock
   INPUT incoming_quantity
   INPUT dispatched_quantity
   INPUT actual_closing_stock
   INPUT target_stock_level
   expected_closing_stock =
      opening_stock + incoming_quantity - dispatched_quantity
   DISPLAY expected_closing_stock
   IF actual_closing_stock = expected_closing_stock THEN
      DISPLAY "No Discrepancy"
   ELSE IF actual_closing_stock < expected_closing_stock THEN
      shortage = expected_closing_stock - actual_closing_stock
      DISPLAY shortage
  ELSE
      surplus = actual_closing_stock - expected_closing_stock
      DISPLAY surplus
   END IF
   replenishment_quantity =
      MAXIMUM(0, target_stock_level - actual_closing_stock)
   DISPLAY replenishment_quantity
END FOR
STOP
```

7.source code

Warehouse-Inventory-Reconciliation.py | ATM-Transaction-Sequence-Simulation.PY

Warehouse-Inventory-Reconciliation.py > ...

```python
number_of_products = int(input("Enter the number of products: "))

for i in range(number_of_products):
    print("\nProduct", i + 1)

    product_name = input("Enter product name: ")
    opening_stock = int(input("Enter opening stock: "))
    incoming_quantity = int(input("Enter incoming quantity: "))
    dispatched_quantity = int(input("Enter dispatched quantity: "))
    actual_closing_stock = int(input("Enter actual closing stock: "))
    target_stock_level = int(input("Enter target stock level: "))

    expected_closing_stock = (opening_stock + incoming_quantity - dispatched_quantity)

    print("\nProduct Name:", product_name)
    print("Expected Closing Stock:", expected_closing_stock)
    print("Actual Closing Stock:", actual_closing_stock)

    if actual_closing_stock == expected_closing_stock:
        print("Status: No Discrepancy")

    elif actual_closing_stock < expected_closing_stock:
        shortage = expected_closing_stock - actual_closing_stock
        print("Status: Shortage")
        print("Shortage Quantity:", shortage)

    else:
        surplus = actual_closing_stock - expected_closing_stock
        print("Status: Surplus")
        print("Surplus Quantity:", surplus)

    replenishment_quantity = max(0, target_stock_level - actual_closing_stock)

    print("Target Stock Level:", target_stock_level)
    print("Replenishment Quantity:", replenishment_quantity)

print("\nInventory reconciliation completed.")
```

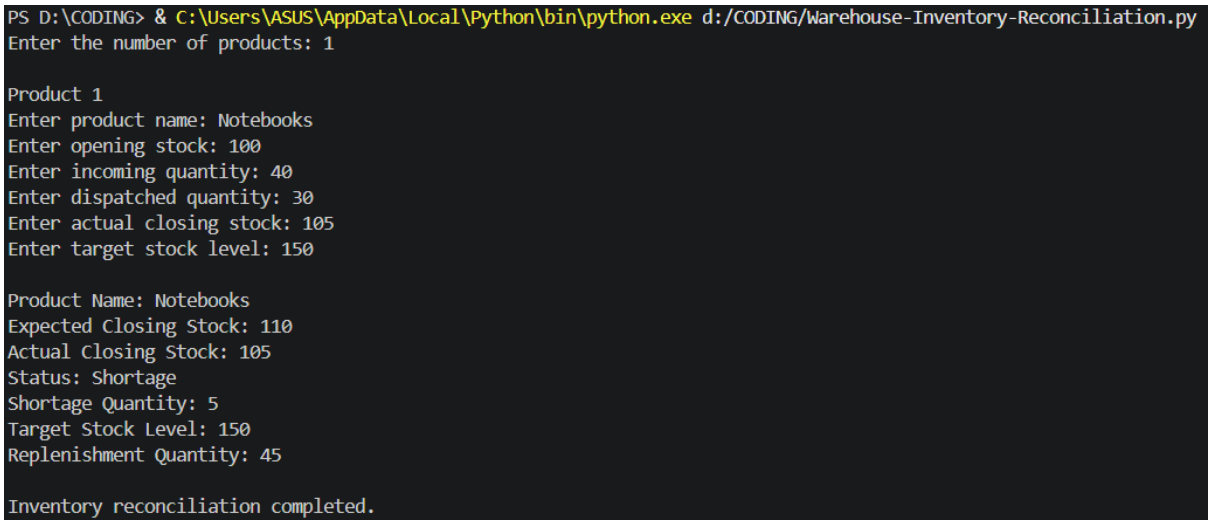
8.**Test Cases**

Test Case	Product	Opening	Incoming	Dispatched	Actual Closing	Target	Expected Result
TC1	Notebooks	100	40	30	110	150	No discrepancy; replenishment 40
TC2	Pens	200	50	80	160	250	Shortage 10; replenishment 90
TC3	Pencils	150	20	40	135	180	No discrepancy; replenishment 45
TC4	Erasers	100	30	20	120	100	Surplus 10; replenishment 0

9. Output Screenshots

The output screenshot demonstrates the execution of the Warehouse Inventory Reconciliation Python program. It displays the product details, expected closing stock, actual closing stock, discrepancy status, and replenishment quantity.

Screenshot 1: Successful execution of the Warehouse Inventory Reconciliation program.



10. Conclusion

The Warehouse Inventory Reconciliation project was implemented using Python. The program calculates expected closing stock from opening stock, incoming quantities, and dispatched quantities. It compares expected stock with actual stock to identify shortages, surpluses, or no discrepancies. It also calculates the replenishment quantity required to reach the target stock level. This project demonstrates the practical use of variables, user input, arithmetic operators, conditional statements, and loops in inventory management.